

Fəsil 17. Autoencoders, GANs və Diffusion Modelləri

Autoencoders, giriş məlumatlarının sıx təsvirlərini öyrənə bilən süni neyron şəbəkələridir. Bu təsvirlər *latent representations* və ya *codings* adlanır və heç bir nəzarət olmadan (yəni, təlim seti etiketsizdir) öyrənilir. Bu codings adətən giriş məlumatlarından çox daha aşağı ölçülüyə malikdir, bu da autoencoders-i dimensionality reduction (bax Fəsil 8) üçün, xüsusən vizuallaşdırma məqsədləri üçün faydalı edir. Autoencoders həmçinin feature detectors kimi çıxış edir və dərin neyron şəbəkələrinin unsupervised pretraining-i üçün istifadə edilə bilər (Fəsil 11-də müzakirə etdiyimiz kimi). Nəhayət, bəzi autoencoders *generative modellər* dir: onlar təlim məlumatlarına çox bənzəyən yeni məlumatları təsadüfi olaraq yarada bilirlər. Məsələn, siz üz şəkilləri üzərində bir autoencoder öyrədə bilərsiniz və o, daha sonra yeni üzlər yarada bilər.

Generative adversarial networks (GANs) da məlumat yarada bilən neyron şəbəkələrdir. Əslində, onlar o qədər inandırıcı üz şəkilləri yarada bilirlər ki, təmsil etdikləri insanların mövcud olmadığına inanmaq çətinidir. Siz bunu <https://thispersondoesnotexist.com> saytında özünüz görə bilərsiniz, bu sayt StyleGAN adlanan bir GAN arxitekturası tərəfindən yaradılan üzləri göstərir. Həmçinin <https://thisrentaldoesnotexist.com> saytında bəzi yaradılmış Airbnb elanlarını görə bilərsiniz. GANs indi super resolution (şəklin ayırd etmə qabiliyyətini artırmaq), rəngləndirmə, güclü şəkil redaktəsi (məsələn, foto bombalarını realistik fonla əvəz etmək), sadə eskizləri fotorealistik şəkillərə çevirmək, videoda növbəti kadrları proqnozlaşdırmaq, dataset-i artırmaq (digər modelləri öyrətmək üçün), digər növ məlumatları (mətn, audio və zaman seriyaları kimi) yaratmaq, digər modellərin zəif tərəflərini müəyyən edərək onları gücləndirmək və daha çoxu üçün geniş istifadə olunur.

Generative learning partiyasına daha yeni bir əlavə *diffusion modelləri* dir. 2021-ci ildə onlar GAN-lardan daha müxtəlif və daha yüksək keyfiyyətli şəkillər yaratmağı bacardılar, eyni zamanda təlim etmək daha asan idi. Bununla belə, diffusion modelləri işləmək üçün daha yavaşıdır.

Autoencoders, GANs və diffusion modellərinin hamısı unsupervised-dir, hamısı latent təsvirləri öyrənir, hamısı generative modellər kimi istifadə edilə bilər və bir çox oxşar tətbiqlərə malikdirlər. Lakin onlar çox fərqli işləyirlər:

- Autoencoders sadəcə girişlərini çıxışlarına köçürməyi öyrənirlər. Bu, əhəmiyyətsiz bir tapşırıq kimi səslənə bilər, lakin görəcəyiniz kimi, şəbəkəni müxtəlif yollarla məhdudlaşdırmaq onu olduqca çətinləşdirə bilər. Məsələn, latent təsvirlərin ölçüsünü məhdudlaşdırma bilərsiniz və ya girişlərə səs-küy əlavə edib şəbəkəni orijinal girişləri bərpa etmək üçün öyrədə bilərsiniz. Bu məhdudiyyətlər autoencoder-in girişləri birbaşa çıxışlara köçürməsinin

qarşısını alır və onu məlumatları təmsil etməyin səmərəli yollarını öyrənməyə məcbur edir. Qısacası, codings autoencoder-in bəzi məhdudiyyətlər altında eynilik funksiyasını öyrənməsinin yan məhsullarıdır.

- GANs iki neyron şəbəkədən ibarətdir: təlim məlumatlarına bənzər məlumatlar yaratmağa çalışan bir *generator* və həqiqi məlumatları saxtadan ayırmağa çalışan bir *discriminator*. Bu arxitektura deep learning-də çox orijinaldır, çünki generator və discriminator təlim zamanı bir-biri ilə rəqabət aparır: generator tez-tez realistik saxta pul düzəltməyə çalışan cinayətkar, discriminator isə həqiqi pulu saxtadan ayırmağa çalışan polis müstəntiqi kimi müqayisə olunur. *Adversarial training* (rəqabət edən neyron şəbəkələrin təlimi) 2010-cu illərin ən mühüm yeniliklərindən biri hesab olunur. 2016-cı ildə Yann LeCun həтта bunun "maşın öyrənməsində son 10 ilin ən maraqlı ideyası" olduğunu söyləmişdir.
- Bir *denoising diffusion probabilistic model* (DDPM) bir şəkildən çox kiçik bir miqdar səs-küyü çıxarmaq üçün öyrədilir. Əgər siz tamamilə Gaussian səs-küylə dolu bir şəkil götürüb, diffusion modelini həmin şəkil üzərində təkrar-təkrar işlətsəniz, təlim şəkillərinə bənzər (lakin eyni olmayan) yüksək keyfiyyətli bir şəkil tədricən ortaya çıxacaq.

Bu fəsildə biz autoencoders-lərin necə işlədiyini və onları dimensionality reduction, feature extraction, unsupervised pretraining və ya generative modellər kimi necə istifadə edəcəyimizi daha dərinədən araşdırmaqla başlayacağıq. Bu, bizi təbii olaraq GAN-lara gətirəcək. Saxta şəkillər yaratmaq üçün sadə bir GAN quracağıq, lakin təlimin çox vaxt olduqca çətin olduğunu görəcəyik. Adversarial training zamanı qarşılaşacağınız əsas çətinlikləri, həmçinin bu çətinliklərin öhdəsindən gəlmək üçün bəzi əsas texnikaları müzakirə edəcəyik. Və nəhayət, bir DDPM quracağıq, öyrədəcəyik və ondan şəkillər yaratmaq üçün istifadə edəcəyik. Autoencoders ilə başlayaq!

Səmərəli Məlumat Təsvirləri

Aşağıdakı ədəd ardıcılıqlarından hansını əzbərləmək sizə daha asan gəlir?

40, 27, 25, 36, 81, 57, 10, 73, 19, 68

50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14

İlk baxışda, birinci ardıcılıq daha asan görünür, çünki daha qısadır. Lakin ikinci ardıcılığa diqqətlə baxsanız, onun 50-dən 14-ə qədər cüt ədədlərin siyahısı olduğunu görürsünüz. Bu naxışı gördükdən sonra, ikinci ardıcılıq birincidən daha asan əzbərlənir, çünki yalnız naxışı (yəni azalan cüt ədədlər) və başlanğıc və son ədədləri (yəni 50 və 14) xatırlamaq lazımdır. Qeyd edək ki, əgər çox uzun ardıcılıqları tez və asanlıqla əzbərləyə bilsəydiniz, ikinci ardıcılıqda naxışın mövcudluğuna o qədər də əhəmiyyət verməzdiniz. Hər bir ədədi əzbərləyərdiniz, bu da bu qədər. Uzun ardıcılıqları

əzbərləməyin çətin olması, naxışları tanımağın faydalı olmasını təmin edir və ümid edirəm ki, bu, autoencoder-i təlim zamanı məhdudlaşdırmağın onu məlumatlardakı naxışları aşkar etməyə və istifadə etməyə itələməsini aydınlaşdırır.

Yaddaş, qavrayış və naxış uyğunlaşdırması arasındakı əlaqə 1970-ci illərin əvvəllərində William Chase və Herbert Simon tərəfindən məşhur şəkildə öyrənilmişdir. Onlar müşahidə etdilər ki, ekspert şahmatçılar oyun taxtasına cəmi beş saniyə baxaraq bütün fiqurların yerlərini əzbərləyə bilirdilər, bu, əksər insanlar üçün qeyri-mümkün bir işdir. Lakin bu, yalnız fiqurlar realistik mövqelərdə (faktiki oyunlardan) yerləşdirildikdə mümkün idi, təsadüfi yerləşdirildikdə yox. Şahmat ekspertlərinin sizdən və məndən daha yaxşı yaddaşı yoxdur; onlar sadəcə oyuna olan təcrübələri sayəsində şahmat naxışlarını daha asan görürlər. Naxışları görmək məlumatı səmərəli saxlamağa kömək edir.

Bu yaddaş təcrübəsindəki şahmatçılar kimi, bir autoencoder da giriş məlumatlarında naxışları axtarır və tapır. O, bu naxışları istifadə edərək məlumatları səmərəli şəkildə təmsil etməyi öyrənir.

[Şəkil 17-1] – Autoencoder: girişlər encoder-dən keçərək codings (latent təsvir) yaradılır, sonra decoder kodları yenidən girişlərə bənzər bir çıxışa çevirir.

Çıxışlar tez-tez *reconstructions* adlanır, çünki autoencoder girişləri yenidən qurmağa çalışır. Məsrəf funksiyası, rekonstruksiyalar girişlərdən fərqli olduqda modeli cəzalandıran bir *reconstruction loss* ehtiva edir.

Daxili təsvir giriş məlumatlarından daha aşağı ölçülülüyə malik olduğu üçün (2D əvəzinə 3D), autoencoder *undercomplete* (tamamlanmamış) adlanır. Bir undercomplete autoencoder öz girişlərini codings-ə trivially köçürə bilməz, lakin girişlərinin surətini çıxarmaq üçün bir yol tapmalıdır. O, giriş məlumatlarındakı ən mühüm xüsusiyyətləri öyrənməyə (və əhəmiyyətsiz olanları atmağa) məcburdur.

Gəlin dimensionality reduction üçün çox sadə bir undercomplete autoencoder-in necə tətbiq ediləcəyini görək.

Undercomplete Linear Autoencoder ilə PCA-nın yerinə yetirilməsi

Əgər autoencoder yalnız xətti aktivasiyalardan istifadə edirsə və məsrəf funksiyası mean squared error (MSE)-dirsə, o, principal component analysis (PCA; bax Fəsil 8) i yerinə yetirir.

Aşağıdakı kod 3D dataset üzərində PCA həyata keçirmək üçün sadə bir xətti autoencoder qurur və onu 2D-yə layihələndirir:

```
import tensorflow as tf

encoder = tf.keras.Sequential([tf.keras.layers.Dense(2)])
decoder = tf.keras.Sequential([tf.keras.layers.Dense(3)])
autoencoder = tf.keras.Sequential([encoder, decoder])

optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)

autoencoder.compile(loss="mse", optimizer=optimizer)
```

Bu kod əvvəlki fəsillərdə qurduğumuz MLP-lərdən həqiqətən çox fərqli deyil, lakin bir neçə şeyi qeyd etmək lazımdır:

- Biz autoencoder-i iki alt komponentə ayırdıq: encoder və decoder. Hər ikisi tək Dense qatı olan adi Sequential modellərdir və autoencoder encoder və ardınca decoder-i ehtiva edən bir Sequential modeldir (xatırlayın ki, bir model başqa bir modeldə qat kimi istifadə edilə bilər).
- Autoencoder-in çıxışlarının sayı girişlərin sayına bərabərdir (yəni 3).
- PCA həyata keçirmək üçün heç bir aktivasiya funksiyası istifadə etmirik (yəni bütün neyronlar xəttidir) və məsrəf funksiyası MSE-dir. Çünki PCA xətti transformasiyadır. Daha mürəkkəb və qeyri-xətti autoencoders-ləri qısa müddətdə görəcəyik.

İndi modeli Fəsil 8-də istifadə etdiyimiz eyni sadə yaradılmış 3D dataset üzərində öyrədək.

[Şəkil 17-2] – 3D təlim seti və onun 2D codings-i. Autoencoder PCA-ya bənzər bir layihə tapır.

Stacked Autoencoders

Digər müzakirə etdiyimiz neyron şəbəkələr kimi, autoencoders da çoxsaylı gizli qatlara malik ola bilər. Bu halda onlar *stacked autoencoders* (və ya *deep autoencoders*) adlanır. Daha çox qat əlavə etmək autoencoder-ə daha mürəkkəb codings öyrənməyə kömək edir. Bununla belə, autoencoder-i çox güclü etməmək üçün diqqətli olmaq lazımdır. Elə bir güclü encoder təsəvvür edin ki, hər bir girişi tək bir ixtiyari ədədə xəritələşdirməyi öyrənir (və decoder əks xəritələşdirməni öyrənir). Aydın ki, belə bir autoencoder təlim məlumatlarını mükəmməl şəkildə yenidən quracaq, lakin bu prosesdə heç bir faydalı məlumat təsviri öyrənməyəcək və yeni instanslara yaxşı ümumiləşdirmə ehtimalı azdır.

Stacked autoencoder-in arxitekturası adətən mərkəzi gizli qata (coding layer) nisbətən simmetrik olur. Sadə desək, o, sendviçə bənzəyir. Məsələn, Fashion MNIST (Fəsil 10-da təqdim edilmişdir) üçün bir autoencoder 784 girişə, ardınca 100 neyronlu bir gizli qata, sonra 30 neyronlu mərkəzi gizli qata, sonra 100 neyronlu başqa bir gizli qata və 784 neyronlu çıxış qatına malik ola bilər. Bu stacked autoencoder Şəkil 17-3-də təsvir edilmişdir.

[Şəkil 17-3] – Stacked autoencoder arxitekturası: 784 giriş, 100 gizli, 30 coding, 100 gizli, 784 çıxış.

Keras ilə Stacked Autoencoder-in Tətbiqi

Siz stacked autoencoder-i adi bir dərin MLP kimi tətbiq edə bilərsiniz:

```
stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
stacked_ae = tf.keras.Sequential([stacked_encoder, stacked_decoder])

stacked_ae.compile(loss="mse", optimizer="nadam")
history = stacked_ae.fit(X_train, X_train, epochs=20,
                          validation_data=(X_valid, X_valid))
```

Bu kodu nəzərdən keçirək:

- Əvvəlki kimi, autoencoder modelini iki alt modelə ayırdıq: encoder və decoder.
- Encoder 28×28 piksellə bozqanadlı şəkilləri qəbul edir, onları düzləşdirir ki, hər bir şəkil 784 ölçülü vektor kimi təmsil olunsun, sonra bu vektorları iki Dense qatından (azalan ölçülərlə: 100 vahid, sonra 30 vahid) keçirir, hər ikisi ReLU aktivasiya funksiyasından istifadə edir. Hər bir giriş şəkli üçün encoder 30 ölçülü bir vektor çıxarır.
- Decoder encoder tərəfindən çıxarılan 30 ölçülü codings-i qəbul edir və onları iki Dense qatından (artan ölçülərlə: 100 vahid, sonra 784 vahid) keçirir və son vektorları 28×28 massivlərə yenidən formalaşdırır ki, decoder-in çıxışları encoder-in girişləri ilə eyni formaya malik olsun.

- Stacked autoencoder-i kompilyasiya edərkən MSE loss və Nadam optimallaşdırmasından istifadə edirik.

Nəhayət, modeli X_{train} -i həm giriş, həm də hədəf kimi istifadə edərək öyrədirik. Eynilə, X_{valid} -i həm validasiya girişləri, həm də hədəflər kimi istifadə edirik.

[Şəkil 17-4] – Orijinal şəkillər və stacked autoencoder tərəfindən yenidən qurulmuş şəkillər. Rekonstruksiyalar bir qədər bulanıqdır, lakin əsas xüsusiyyətləri saxlayır.

Hazırda bu modellə davam edək.

Fashion MNIST Dataset-inin Vizuallaşdırılması

İndi stacked autoencoder-i öyrətdiyimizə görə, onu dataset-in ölçülülüyünü azaltmaq üçün istifadə edə bilərik. Vizuallaşdırma üçün, bu, digər dimensionality reduction alqoritmləri ilə (məsələn, Fəsil 8-də müzakirə etdiklərimiz) müqayisədə çox yaxşı nəticələr vermir, lakin autoencoders-in böyük üstünlüyü onların çoxsaylı instansları və çoxsaylı xüsusiyyətləri olan böyük dataset-ləri idarə edə bilməsidir. Beləliklə, bir strategiya, autoencoder istifadə edərək ölçülülüyü ağlabatan bir səviyyəyə endirmək, sonra vizuallaşdırma üçün başqa bir dimensionality reduction alqoritmindən istifadə etməkdir. Gəlin Fashion MNIST-i vizuallaşdırmaq üçün bu strategiyayı istifadə edək. Əvvəlcə stacked autoencoder-imizin encoder-indən istifadə edərək ölçülülüyü 30-a endirəcəyik, sonra Scikit-Learn-ın t-SNE alqoritminin tətbiqindən istifadə edərək ölçülülüyü 2-yə endirib vizuallaşdıracağıq:

```
from sklearn.manifold import TSNE
```

```
X_valid_compressed = stacked_encoder.predict(X_valid)
tsne = TSNE(init="pca", learning_rate="auto", random_state=42)
```

```
X_valid_2D = tsne.fit_transform(X_valid_compressed)
```

İndi dataset-i çəkə bilərik:

```
plt.scatter(X_valid_2D[:, 0], X_valid_2D[:, 1], c=y_valid, s=10,
            cmap="tab10")
```

```
plt.show()
```

Şəkil 17-5 nəticədə yaranan səpələnmə qrafikini, bəzi şəkilləri göstərməklə bir az gözəlləşdirilmiş şəkildə göstərir. t-SNE alqritmi siniflərlə kifayət qədər yaxşı uyğunlaşan bir neçə cluster müəyyən etdi (hər bir sinif fərqli bir rənglə təmsil olunur).

[Şəkil 17-5] – t-SNE ilə vizuallaşdırılmış Fashion MNIST validasiya seti, autoencoder ilə 30 ölçüyə endirilmişdir.

Stacked Autoencoders istifadə edərək Unsupervised Pretraining

Fəsil 11-də müzakirə etdiyimiz kimi, əgər mürəkkəb bir supervised tapşırıqla məşğul olursunuzsa, lakin çox az etiketli təlim məlumatınız varsa, bir həll, oxşar bir tapşırığı yerinə yetirən bir neyron şəbəkəsi tapmaq və onun aşağı qatlarını təkrar istifadə etməkdir. Bu, az təlim məlumatından istifadə edərək yüksək performanslı bir model öyrətməyə imkan verir, çünki neyron şəbəkəniz bütün aşağı səviyyəli xüsusiyyətləri öyrənmək məcburiyyətində deyil; o, sadəcə mövcud şəbəkə tərəfindən öyrənilmiş feature detectors-ləri təkrar istifadə edəcək.

Eynilə, əgər böyük bir dataset-iniz varsa, lakin onun əksəriyyəti etiketsizdirsə, əvvəlcə bütün məlumatlardan istifadə edərək bir stacked autoencoder öyrədə bilərsiniz, sonra faktiki tapşırığınız üçün bir neyron şəbəkəsi yaratmaq üçün aşağı qatları təkrar istifadə edə bilərsiniz və onu etiketli məlumatlardan istifadə edərək öyrədə bilərsiniz. Məsələn, Şəkil 17-6 stacked autoencoder-dən bir təsnifat neyron şəbəkəsi üçün unsupervised pretraining etmək üçün necə istifadə edəcəyini göstərir. Təsnifatçıyı öyrədərkən, əgər həqiqətən çox az etiketli təlim məlumatınız varsa, əvvəlcədən öyrədilmiş qatları (heç olmasa aşağı olanları) dondurmaq istəyə bilərsiniz.

[Şəkil 17-6] – Unsupervised pretraining: Mərhələ 1: Bütün məlumatlardan istifadə edərək autoencoder-i öyrədin. Mərhələ 2: Autoencoder-in aşağı qatlarını təkrar istifadə edərək təsnifatçı qurun və onu etiketli məlumatlarda öyrədin.

Sonra, stacked autoencoders-i öyrətmək üçün bir neçə texnikaya baxaq.

Çəkilərin Bağlanması (Tying Weights)

Bir autoencoder səliqəli simmetrik olduqda, bizim indi qurduğumuz kimi, ümumi bir texnika decoder qatlarının çəkilərini encoder qatlarının çəkilərinə *bağlamaq* (tie) dır. Bu, modeldəki çəkilərin sayını yarıya endirir, təlimi sürətləndirir və overfitting riskini məhdudlaşdırır. Xüsusilə, əgər autoencoder ümumilikdə N qata malikdirsə (giriş qatını saymasaq) və WL L ci qatın birləşmə çəkilərini təmsil edirsə (məsələn, qat 1 ilk gizli qatdır, qat $N/2$ coding qatıdır və qat N çıxış qatıdır), onda decoder qat çəkiləri $WL = WN - L + 1$ ($L = N/2 + 1, \dots, N$ ilə) olaraq təyin edilə bilər.

Keras istifadə edərək qatlar arasında çəkiləri bağlamaq üçün, gəlin xüsusi bir qat təyin edək:

```
class DenseTranspose(tf.keras.layers.Layer):
```

```

def __init__(self, dense, activation=None, **kwargs):
    super().__init__(**kwargs)
    self.dense = dense
    self.activation = tf.keras.activations.get(activation)
def build(self, batch_input_shape):
    self.biases = self.add_weight(name="bias",
                                  shape=self.dense.input_shape[-1],
                                  initializer="zeros")

    super().build(batch_input_shape)
def call(self, inputs):
    Z = tf.matmul(inputs, self.dense.weights[0], transpose_b=True)

    return self.activation(Z + self.biases)

```

Bu xüsusi qat adi Dense qatı kimi davranır, lakin başqa bir Dense qatının çəkirlərindən, transpozisiya edilmiş şəkildə istifadə edir (`transpose_b=True` təyin etmək ikinci argumenti transpozisiya etməyə bərabərdir, lakin daha səmərəlidir, çünki transpozisiyanı `matmul` əməliyyatı daxilində yerinə yetirir). Lakin o, öz bias vektorundan istifadə edir. İndi əvvəlki kimi, lakin decoder-in Dense qatları encoder-in Dense qatlarına bağlanmış yeni bir stacked autoencoder qura bilərik:

```

dense_1 = tf.keras.layers.Dense(100, activation="relu")
dense_2 = tf.keras.layers.Dense(30, activation="relu")

tied_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    dense_1,
    dense_2
])
tied_decoder = tf.keras.Sequential([
    DenseTranspose(dense_2, activation="relu"),
    DenseTranspose(dense_1),
    tf.keras.layers.Reshape([28, 28])
])

tied_ae = tf.keras.Sequential([tied_encoder, tied_decoder])

```

Bu model, əvvəlki model ilə təxminən eyni rekonstruksiya səhvinə nail olur, demək olar ki, yarısı qədər parametr istifadə edir.

Qat-qat Pretraining

Əgər təlim məlumatları çox böyük deyilsə, bütün stacked autoencoder-i bir dəfəyə öyrətmək yaxşı işləyir. Lakin çox dərin stacked autoencoders üçün, onları bir qat bir

qat öyrətmək daha yaxşı ola bilər. Məsələn, əvvəlcə tək gizli qatlı autoencoder-i (birinci qat) öyrədin, sonra onu dondurun, növbəti qatı əlavə edin və s. Bu, 2006-cı ildə Geoffrey Hinton və başqalarının kəşfi ilə deep learning dalğasının tetikleyicilərindən biri idi. Onlar restricted Boltzmann machines (RBMs) istifadə etdilər, lakin 2007-ci ildə Yoshua Bengio və başqaları autoencoders-in eyni dərəcədə yaxşı işlədiyini göstərdilər.

Autoencoders yalnız sıx şəbəkələrlə məhdudlaşmır: siz həmçinin convolutional autoencoders qura bilərsiniz. İndi onlara baxaq.

Convolutional Autoencoders

Əgər şəkillərlə işləyirsinizsə, indiyə qədər gördüyümüz autoencoders yaxşı işləməyəcək (şəkillər çox kiçik olmadıqca): Fəsil 14-də gördüyünüz kimi, convolutional neural networks sıx şəbəkələrdən şəkillərlə işləmək üçün daha uyğundur. Beləliklə, əgər şəkillər üçün bir autoencoder qurmaq istəyirsinizsə (məsələn, unsupervised pretraining və ya dimensionality reduction üçün), *convolutional autoencoder* qurmalısınız. Encoder, convolutional qatlar və pooling qatlarından ibarət adi bir CNN-dir. O, adətən girişlərin məkan ölçülülüyünü (hündürlük və en) azaldır, eyni zamanda dərinliyi (feature map-ların sayı) artırır. Decoder əksini etməlidir (şəkli böyütmək və dərinliyi orijinal ölçülərə qaytarmaq) və bunun üçün transpose convolutional qatlarından istifadə edə bilərsiniz (alternativ olaraq, upsampling qatlarını convolutional qatlarla birləşdirə bilərsiniz). Budur Fashion MNIST üçün əsas bir convolutional autoencoder:

```
conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 14 x 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 7 x 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 3 x 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAvgPool2D() # output: 30
])

conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"),
```

```
tf.keras.layers.Reshape([28, 28])
])

conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])
```

RNN-lər kimi digər arxitektura növləri ilə də autoencoders yaratmaq mümkündür (nümunə üçün notebook-a baxın).

Yaxşı, bir anlıq geri çəkilib nəzər salaq. İndiyə qədər biz müxtəlif növ autoencoders-lərə (əsas, stacked və convolutional) və onların necə öyrədilməsinə (bir dəfəyə və ya qat-qat) baxdıq. Biz həmçinin bir neçə tətbiqə: məlumat vizuallaşdırması və unsupervised pretraining-ə baxdıq.

İndiyə qədər, autoencoder-i maraqlı xüsusiyyətlər öyrənməyə məcbur etmək üçün coding qatının ölçüsünü məhdudlaşdırıb onu undercomplete etdik. Əslində, istifadə edilə bilən bir çox başqa məhdudiyyətlər var, o cümlədən coding qatının girişlər qədər böyük, hətta daha böyük olmasına imkan verənlər, nəticədə *overcomplete autoencoder* yaranır. Sonrakı bölmələrdə bir neçə daha növ autoencoder-ə baxacağıq: denoising autoencoders, sparse autoencoders və variational autoencoders.

Denoising Autoencoders

Autoencoder-i faydalı xüsusiyyətlər öyrənməyə məcbur etməyin başqa bir yolu, onun girişlərinə səs-küy əlavə etmək və onu orijinal, səs-küysüz girişləri bərpa etmək üçün öyrətməkdir. Bu fikir 1980-ci illərdən bəri mövcuddur (məsələn, Yann LeCun-un 1987-ci il magistr dissertasiyasında qeyd olunur). 2008-ci ildə Pascal Vincent və digərlərinin məqaləsində autoencoders-in feature extraction üçün də istifadə edilə biləcəyi göstərildi. 2010-cu ildə Vincent və digərləri *stacked denoising autoencoders* təqdim etdilər.

Səs-küy, girişlərə əlavə edilən təmiz Gaussian səs-küyü və ya təsadüfi söndürülmüş girişlər (dropout kimi, Fəsil 11-də təqdim edilmişdir) ola bilər. Şəkil 17-8 hər iki variantı göstərir.

Tətbiq sadədir: o, encoder-in girişlərinə tətbiq edilən əlavə Dropout qatı olan adi bir stacked autoencoder-dir (və ya GaussianNoise qatından istifadə edə bilərsiniz). Xatırlayın ki, Dropout qatı yalnız təlim zamanı aktivdir (və GaussianNoise qatı da belədir):

```
dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(100, activation="relu"),
```

```

        tf.keras.layers.Dense(30, activation="relu")
    ])
dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])

dropout_ae = tf.keras.Sequential([dropout_encoder, dropout_decoder])

```

[Şəkil 17-8] – Denoising autoencoder: sola Gaussian səs-küyü, sağa dropout.

[Şəkil 17-9] – Orijinal şəkillər, səs-küylü şəkillər və denoising autoencoder tərəfindən bərpa edilmiş şəkillər.

Sparse Autoencoders

Tez-tez yaxşı feature extraction-a gətirən başqa bir məhdudiyyət *sparsity* (seyreklik) dir: məsrəf funksiyasına müvafiq bir termin əlavə etməklə, autoencoder coding qatındakı aktiv neyronların sayını azaltmağa itəlmir. Məsələn, orta hesabla coding qatında yalnız 5% əhəmiyyətli dərəcədə aktiv neyron olmasına itələyə bilər. Bu, autoencoder-i hər bir girişi az sayda aktivasiyaların kombinasiyası kimi təmsil etməyə məcbur edir. Nəticədə, coding qatındakı hər bir neyron adətən faydalı bir xüsusiyyəti təmsil edir (əgər ayda yalnız bir neçə söz danışa bilsəydiniz, yəqin ki, onları dinləməyə dəyər etməyə çalışdınız).

Sadə bir yanaşma, coding qatında sigmoid aktivasiya funksiyasından istifadə etmək (codings-i 0 ilə 1 arasında məhdudlaşdırmaq üçün), böyük bir coding qatı (məsələn, 300 vahid) istifadə etmək və coding qatının aktivasiyalarına bəzi ℓ_1 regularizasiyası əlavə etməkdir. Decoder sadəcə adi bir decoder-dir:

```

sparse_l1_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid"),
    tf.keras.layers.ActivityRegularization(l1=1e-4)
])
sparse_l1_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])

sparse_l1_ae = tf.keras.Sequential([sparse_l1_encoder, sparse_l1_decoder])

```

Bu ActivityRegularization qatı sadəcə girişlərini qaytarır, lakin yan təsir olaraq girişlərinin mütləq dəyərlərinin cəminə bərabər bir təlim loss-u əlavə edir. Bu, yalnız təlimə təsir edir. Ekvivalent olaraq, ActivityRegularization qatını çıxarıb əvvəlki qatda `activity_regularizer=tf.keras.regularizers.l1(1e-4)` təyin edə bilərsiniz. Bu cəza neyron şəbəkəni sıfıra yaxın codings istehsal etməyə təşviq edəcək, lakin girişləri düzgün bərpa etməsə, cəzalandırılacağı üçün, ən azı bir neçə sıfırdan fərqli dəyər çıxarmalı olacaq. ℓ_1 normasından istifadə etmək, ℓ_2 normasından fərqli olaraq, neyron şəbəkəni ən vacib codings-ləri qoruyarkən, giriş şəkli üçün lazım olmayanları aradan qaldırmağa itələyəcək (sadəcə bütün codings-ləri azaltmaq əvəzinə).

Daha yaxşı nəticələr verən başqa bir yanaşma, hər bir təlim iterasiyasında coding qatının faktiki sparsity-sini ölçmək və ölçülmüş sparsity hədəf sparsity-dən fərqləndikdə modeli cəzalandırmaqdır. Bunu etmək üçün, bütün təlim batch-i üzərində coding qatındakı hər bir neyronun ortalama aktivasiyasını hesablayırıq. Batch ölçüsü çox kiçik olmamalıdır, əks halda ortalama dəqiq olmayacaq.

Hər bir neyron üçün ortalama aktivasiyanı əldə etdikdən sonra, çox aktiv və ya kifayət qədər aktiv olmayan neyronları məsrəf funksiyasına bir *sparsity loss* əlavə etməklə cəzalandırmaq istəyirik. Məsələn, əgər bir neyronun ortalama aktivasiyası 0.3 olduğunu ölçsək, lakin hədəf sparsity 0.1-dirsə, daha az aktiv olmaq üçün cəzalandırılmalıdır. Bir yanaşma, sadəcə kvadrat səhvi $(0.3 - 0.1)^2$ məsrəf funksiyasına əlavə etmək ola bilər, lakin praktikada daha yaxşı bir yanaşma Kullback–Leibler (KL) divergence (Fəsil 4-də qısaca müzakirə edilmişdir) istifadə etməkdir, bu, ortalama kvadrat səhvdən daha güclü qradientlərə malikdir, Şəkil 17-10-da görə bilərsiniz.

[Şəkil 17-10] – KL divergence, MAE və MSE-nin müqayisəsi: KL divergence hədəf dəyər ətrafında daha kəskin qradientlərə malikdir.

Sparsity loss-u çoxaltmaq üçün bir sparsity weight hiperparametri istifadə edə bilərik. Əgər bu çəki çox yüksəkdirsə, model hədəf sparsity-yə sıx bağlanacaq, lakin girişləri düzgün bərpa edə bilməyəcək, modeli faydasız edəcək. Əksinə, çox aşağıdırsa, model sparsity məqsədini əsasən nəzər almayacaq və maraqlı xüsusiyyətlər öyrənməyəcək.

İndi KL divergence əsasında bir sparse autoencoder tətbiq etmək üçün lazım olan hər şeyə sahibik. Əvvəlcə, KL divergence regularizasiyasını tətbiq etmək üçün xüsusi bir regularizator yaradaq:

```
kl_divergence = tf.keras.losses.kullback_leibler_divergence
```

```
class KLDivergenceRegularizer(tf.keras.regularizers.Regularizer):  
    def __init__(self, weight, target):  
        self.weight = weight
```

```

        self.target = target
    def __call__(self, inputs):
        mean_activities = tf.reduce_mean(inputs, axis=0)
        return self.weight * (
            kl_divergence(self.target, mean_activities) +
            kl_divergence(1. - self.target, 1. - mean_activities))

```

İndi coding qatının aktivasiyaları üçün KLDivergenceRegularizer istifadə edərək sparse autoencoder-i qura bilərik:

```

kld_reg = KLDivergenceRegularizer(weight=5e-3, target=0.1)
sparse_kl_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid",
activity_regularizer=kld_reg)
])
sparse_kl_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])

sparse_kl_ae = tf.keras.Sequential([sparse_kl_encoder, sparse_kl_decoder])

```

Bu sparse autoencoder-i Fashion MNIST üzərində öyrətdikdən sonra, coding qatı təxminən 10% sparsity-yə malik olacaq.

Variational Autoencoders

Autoencoders-in mühüm bir kateqoriyası 2013-cü ildə Diederik Kingma və Max Welling tərəfindən təqdim edilmişdir və tez bir zamanda ən populyar variantlardan birinə çevrilmişdir: *variational autoencoders* (VAEs).

VAEs, indiyə qədər müzakirə etdiyimiz bütün autoencoders-lərdən olduqca fərqlidir, bu xüsusi yollarla:

- Onlar *probabilistic autoencoders* dır, yəni onların çıxışları qismən təsadüfiliklə müəyyən edilir, hətta təlimdən sonra da (denoising autoencoders-dan fərqli olaraq, onlar təlim zamanı yalnız təsadüfilikdən istifadə edirlər).
- Ən əsası, onlar *generative autoencoders* dır, yəni təlim setindən nümunə götürülmüş kimi görünən yeni instanslar yarada bilirlər.

Bu iki xüsusiyyət VAEs-i RBMs-ə kifayət qədər bənzədir, lakin onları öyrətmək daha asandır və nümunə götürmə prosesi daha sürətlidir (RBMs ilə yeni bir instans nümunəsi götürməzdən əvvəl şəbəkənin "termal tarazlıq" da sabitləşməsini gözləmək lazımdır). Adlarından göründüyü kimi, variational autoencoders variational Bayesian inference yerinə yetirir, bu, təqribi Bayesian inference-in səmərəli bir yoludur. Xatırladaq ki, Bayesian inference, Bayes teoremindən əldə edilən tənlidlərdən istifadə edərək yeni məlumatlara əsaslanaraq ehtimal paylanmasını yeniləmək deməkdir. Orijinal paylanma *prior*, yenilənmiş paylanma isə *posterior* adlanır. Bizim halda, məlumat paylanmasının yaxşı bir təqribini tapmaq istəyirik. Bunu əldə etdikdən sonra, ondan nümunə götürə bilərik.

VAEs-in necə işlədiyinə baxaq. Şəkil 17-11 (sol) bir variational autoencoder-i göstərir. Siz bütün autoencoders-lərin əsas strukturunu tanıya bilərsiniz, bir encoder və ardınca bir decoder (bu nümunədə hər ikisinin iki gizli qatı var), lakin bir fərq var: müəyyən bir giriş üçün birbaşa coding istehsal etmək əvəzinə, encoder bir *mean coding* μ və bir standard deviation σ istehsal edir. Faktiki coding daha sonra Gaussian paylanmasından təsadüfi olaraq nümunə götürülür.

[Şəkil 17-11] – Variational autoencoder (sol) və onun kodlaşdırma fəzası (sağ). Encoder ortalama və standart sapma çıxarır, coding onlardan nümunə götürülür.

Təlim zamanı məsrəf funksiyası (sonra müzakirə ediləcək) codings-ləri tədricən kodlaşdırma fəzası (həmçinin *latent space* adlanır) daxilində miqrasiya etməyə itələyir ki, nəticədə Gaussian nöqtələri buluduna bənzəsin. Bir böyük nəticə ondan ibarətdir ki, bir variational autoencoder-i öyrətdikdən sonra, yeni bir instans yaratmaq çox asandır: sadəcə Gaussian paylanmasından təsadüfi bir coding nümunəsi götürün, onu decode edin və voilà!

İndi məsrəf funksiyasına baxaq. O, iki hissədən ibarətdir. Birincisi, autoencoder-i girişlərini yenidən istehsal etməyə itələyən adi reconstruction loss-dur. Bunun üçün əvvəlki kimi MSE istifadə edə bilərik. İkincisi, autoencoder-i codings-lərinin sadə bir Gaussian paylanmasından nümunə götürülmüş kimi görünməsinə itələyən *latent loss* dur: bu, hədəf paylanma (yəni Gaussian paylanması) ilə codings-lərin faktiki paylanması arasındakı KL divergence-dir. Riyaziyyat sparse autoencoder ilə müqayisədə bir az daha mürəkkəbdir, xüsusən Gaussian səs-küyü səbəbindən, bu, coding qatına ötürülə bilən məlumat miqdarını məhdudlaşdırır. Bu, autoencoder-i faydalı xüsusiyyətlər öyrənməyə itələyir. Xoşbəxtlikdən, tənlidlər sadələşir, beləliklə latent loss Tənlik 17-3 istifadə edərək hesablanı bilər.

Tənlik 17-3. Variational autoencoder-in latent loss-u

$$L = -1/2 \sum_{i=1}^n (1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2)$$

Bu tənləkdə, $\mathcal{L}$ latent loss, n codings-in ölçülülüyü, μ_i və σ_i codings-in i ci komponentinin ortalama və standart sapmasıdır. μ və σ vektorları (bütün μ_i və σ_i -ni ehtiva edir) encoder tərəfindən çıxarılır, Şəkil 17-11 (sol) -də göstərilədiyi kimi.

Variational autoencoder arxitekturasına ümumi bir düzəliş, σ əvəzinə encoder-in $\gamma = \log(\sigma^2)$ çıxarmasıdır. Latent loss daha sonra Tənlək 17-4-də göstərilədiyi kimi hesablanə bilər. Bu yanaşma ədədi cəhətdən daha dayanıqlıdır və təlimi sürətləndirir.

Tənlək 17-4. Variational autoencoder-in latent loss-u, $\gamma = \log(\sigma^2)$ istifadə edilərək yenidən yazılmışdır

$$L = -1/2 \sum_{i=1}^n (1 + \gamma_i - \exp(\gamma_i) - \mu_i^2)$$

Gəlin Fashion MNIST üçün bir variational autoencoder qurmağa başlayaq (Şəkil 17-11-də göstərilədiyi kimi, lakin γ düzəlişi ilə). Əvvəlcə, μ və γ verildikdə codings-i nümunə götürmək üçün xüsusi bir qata ehtiyacımız olacaq:

```
class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs

        return tf.random.normal(tf.shape(log_var)) * tf.exp(log_var / 2) +
mean
```

Bu Sampling qatı iki giriş qəbul edir: mean (μ) və log_var (γ). O, Gaussian paylanmasından təsadüfi bir vektor (γ ilə eyni formada) nümunə götürmək üçün `tf.random.normal()` funksiyasından istifadə edir, ortalama 0 və standart sapma 1 ilə. Sonra onu $\exp(\gamma / 2)$ ilə vurur (bu, σ -yə bərabərdir, riyazi olaraq yoxlaya bilərsiniz) və nəhayət μ əlavə edir və nəticəni qaytarır. Bu, ortalama μ və standart sapma σ olan Gaussian paylanmasından bir codings vektorunu nümunə götürür.

Sonra, functional API-dən istifadə edərək encoder-i yarada bilərik, çünki model tamamilə ardıcıl deyil:

```
codings_size = 10

inputs = tf.keras.layers.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)
codings_mean = tf.keras.layers.Dense(codings_size)(Z) #  $\mu$ 
codings_log_var = tf.keras.layers.Dense(codings_size)(Z) #  $\gamma$ 
codings = Sampling()( [codings_mean, codings_log_var] )
variational_encoder = tf.keras.Model(
    inputs=[inputs],
```

```
outputs=[codings_mean, codings_log_var, codings])
```

Qeyd edək ki, `codings_mean` (μ) və `codings_log_var` (γ) çıxaran Dense qatları eyni girişlərə malikdir (yəni, ikinci Dense qatının çıxışları). Biz sonra həm `codings_mean`, həm də `codings_log_var`-ı Sampling qatına ötürürük. Nəhayət, `variational_encoder` modelinin üç çıxışı var. Yalnız `codings` tələb olunur, lakin biz `codings_mean` və `codings_log_var`-ı da əlavə edirik, əgər onların dəyərlərini yoxlamaq istəsək.

İndi decoder-i quraq:

```
decoder_inputs = tf.keras.layers.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="relu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="relu")(x)
x = tf.keras.layers.Dense(28 * 28)(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)
```

```
variational_decoder = tf.keras.Model(inputs=[decoder_inputs],
outputs=[outputs])
```

Bu decoder üçün, functional API əvəzinə sequential API-dən istifadə edə bilərdik, çünki o, həqiqətən sadə bir qat yığınıdır, indiyə qədər qurduğumuz bir çox decoder-lərlə demək olar ki, eynidir.

Nəhayət, variational autoencoder modelini quraq:

```
_, _, codings = variational_encoder(inputs)
reconstructions = variational_decoder(codings)

variational_ae = tf.keras.Model(inputs=[inputs], outputs=[reconstructions])
```

Biz encoder-in ilk iki çıxışını nəzər almırıq (yalnız `codings`-i decoder-a qidalandırmaq istəyirik). Sonra, latent loss və reconstruction loss-u əlavə etməliyik:

```
latent_loss = -0.5 * tf.reduce_sum(
    1 + codings_log_var - tf.exp(codings_log_var) -
    tf.square(codings_mean),
    axis=-1)

variational_ae.add_loss(tf.reduce_mean(latent_loss) / 784.)
```

Biz əvvəlcə Tənlik 17-4-ü tətbiq edərək batch-dəki hər bir instans üçün latent loss-u hesablayırıq, son ox üzərində cəmləyirik. Sonra batch-dəki bütün instanslar üzərində ortalama loss-u hesablayırıq və nəticəni 784-ə bölürük ki, reconstruction loss ilə

müqayisədə müvafiq miqyasda olsun. Həqiqətən də, variational autoencoder-in reconstruction loss-u piksel rekonstruksiya səhvlərinin cəmi olmalıdır, lakin Keras "mse" loss-u hesablayarkən bütün 784 piksel üzərində ortalama hesablayır, cəm deyil. Beləliklə, reconstruction loss bizə lazım olduğundan 784 dəfə kiçikdir. Biz cəmi hesablamaq üçün xüsusi bir loss təyin edə bilərdik, lakin latent loss-u 784-ə bölmək daha sadədir (son loss lazım olduğundan 784 dəfə kiçik olacaq, lakin bu, sadəcə daha böyük öyrənmə sürəti istifadə etməli olduğumuz deməkdir).

Və nəhayət, autoencoder-i kompilyasiya edib fit edə bilərik!

```
variational_ae.compile(loss="mse", optimizer="nadam")
history = variational_ae.fit(X_train, X_train, epochs=25, batch_size=128,
                             validation_data=(X_valid, X_valid))
```

[Şəkil 17-12] – Variational autoencoder tərəfindən yenidən qurulmuş şəkillər.

[Şəkil 17-13] – Variational autoencoder tərəfindən yaradılmış yeni şəkillər (təsadüfi codings-dən decode edilmişdir).

Generative Adversarial Networks

Generative adversarial networks 2014-cü ildə Ian Goodfellow və digərlərinin məqaləsində təklif edilmişdir və ideya tədqiqatçıları demək olar ki, dərhal həyəcanlandırırsa da, GAN-ların təliminin bəzi çətinliklərini aşmaq bir neçə il çəkdi. Bir çox böyük ideyalar kimi, geriye baxanda sadə görünür: neyron şəbəkələri bir-biri ilə rəqabət aparmağa məcbur edin, ümid edək ki, bu rəqabət onları üstün olmağa itələyəcək. Şəkil 17-14-də göstərildiyi kimi, bir GAN iki neyron şəbəkədən ibarətdir:

Generator

Giriş olaraq təsadüfi bir paylanma (adətən Gaussian) qəbul edir və bəzi məlumatlar – adətən bir şəkil – çıxarır. Siz təsadüfi girişləri yaradılacaq şəklə latent təsvirləri (yəni codings) kimi düşünə bilərsiniz. Beləliklə, gördüyünüz kimi, generator variational autoencoder-dəki decoder ilə eyni funksionallığı təklif edir və yeni şəkillər yaratmaq üçün eyni şəkildə istifadə edilə bilər: ona bir az Gaussian səs-küyü verin və o, tamamilə yeni bir şəkil çıxarır. Lakin o, çox fərqli şəkildə öyrədilir, qısa müddətdə görəcəksiniz.

Discriminator

Generator-dan saxta bir şəkil və ya təlim setindən həqiqi bir şəkil giriş kimi qəbul edir və giriş şəklinin saxta və ya həqiqi olduğunu təxmin etməlidir.

[Şəkil 17-14] – GAN arxitekturası: generator səs-küydən şəkil yaradır, discriminator həqiqi/saxta olduğunu müəyyən edir.

Hər bir təlim iterasiyasında iki mərhələ var. Birinci mərhələdə, biz generator-a Gaussian səs-küyü veririk və bir batch saxta şəkillər istehsal edirik. Sonra bu batch-i eyni sayda həqiqi şəkillərlə birləşdiririk. Həqiqi şəkillər üçün hədəf 1, saxta şəkillər üçün isə 0-dır (onlar saxtadır). Sonra discriminator-u bu batch üzərində öyrədirik. Qeyd edək ki, bu mərhələdə yalnız discriminator öyrədilir; generator-un çəkilişi dəyişdirilmir.

İkinci mərhələdə, generator-a başqa bir batch Gaussian səs-küyü veririk və o, başqa bir batch saxta şəkillər istehsal edir. Yenə discriminator şəkillərin saxta və ya həqiqi olub-olmadığını təyin etmək üçün istifadə olunur. Bu dəfə batch-ə həqiqi şəkillər əlavə etmirik və bütün etikətlər 1 (həqiqi) olaraq təyin edilir: başqa sözlə, biz generator-un discriminator-un (səhvən) həqiqi olduğuna inanacağı şəkillər istehsal etməsini istəyirik! Vacib olaraq, bu mərhələdə discriminator-un çəkilişi dondurulur, beləliklə backpropagation yalnız generator-un çəkilişinə təsir edir.

QEYD

Generator heç vaxt həqiqi şəkilləri görmür, lakin tədricən inandırıcı saxta şəkillər istehsal etməyi öyrənir! Onun aldığı tək şey discriminator vasitəsilə geri axan qradiyentlərdir. Xoşbəxtlikdən, discriminator nə qədər yaxşılaşarsa, bu ikinci əl qradiyentlərdə həqiqi şəkillər haqqında bir o qədər çox məlumat olar, beləliklə generator əhəmiyyətli irəliləyiş edə bilər.

Gəlin Fashion MNIST üçün sadə bir GAN quraq.

Əvvəlcə, generator və discriminator qurmalıyıq. Generator autoencoder-in decoder-inə bənzəyir, discriminator isə adi bir binary classifier-dir: o, bir şəkli giriş kimi qəbul edir və tək vahidli Dense qatı ilə sigmoid aktivasiya funksiyasından istifadə edərək bitir. Hər bir təlim iterasiyasının ikinci mərhələsi üçün, generator və ardınca discriminator-u ehtiva edən tam GAN modelinə də ehtiyacımız var:

```
codings_size = 30
```

```
Dense = tf.keras.layers.Dense
generator = tf.keras.Sequential([
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])
```

```

])
discriminator = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(1, activation="sigmoid")
])

gan = tf.keras.Sequential([generator, discriminator])

```

Sonra, bu modelləri kompilyasiya etməliyik. Discriminator bir binary classifier olduğundan, təbii olaraq binary cross-entropy loss istifadə edə bilərik. Gan modeli də binary classifier-dir, buna görə o da binary cross-entropy loss istifadə edə bilər. Lakin generator yalnız gan modeli vasitəsilə öyrədiləcək, buna görə onu ümumiyyətlə kompilyasiya etməyə ehtiyac yoxdur. Vacib olaraq, discriminator ikinci mərhələdə öyrədilməməlidir, buna görə gan modelini kompilyasiya etməzdən əvvəl onu trainable etmirik:

```

discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
discriminator.trainable = False

```

```

gan.compile(loss="binary_crossentropy", optimizer="rmsprop")

```

QEYD

`trainable` atributu Keras tərəfindən yalnız model kompilyasiya edildikdə nəzərə alınır, buna görə bu kodu işlətdikdən sonra, discriminator-un `fit()` metodu və ya `train_on_batch()` metodu (biz istifadə edəcəyik) çağırıldıqda discriminator *trainable* dır, gan modelində bu metodlar çağırıldıqda isə *trainable* deyil.

Təlim dövrü qeyri-adi olduğundan, adi `fit()` metodundan istifadə edə bilmirik. Bunun əvəzinə, xüsusi bir təlim dövrü yazacağıq. Bunun üçün əvvəlcə şəkillər üzərində iterasiya etmək üçün bir Dataset yaratmalıyıq:

```

batch_size = 32
dataset =
tf.data.Dataset.from_tensor_slices(X_train).shuffle(buffer_size=1000)

dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)

```

İndi təlim dövrünü yazmağa hazırıq. Gəlin onu bir `train_gan()` funksiyasında yerləşdirək:

```

def train_gan(gan, dataset, batch_size, codings_size, n_epochs):

```

```

generator, discriminator = gan.layers
for epoch in range(n_epochs):
    for X_batch in dataset:
        # phase 1 - training the discriminator
        noise = tf.random.normal(shape=[batch_size, codings_size])
        generated_images = generator(noise)
        X_fake_and_real = tf.concat([generated_images, X_batch],
axis=0)

        y1 = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)
        discriminator.train_on_batch(X_fake_and_real, y1)
        # phase 2 - training the generator
        noise = tf.random.normal(shape=[batch_size, codings_size])
        y2 = tf.constant([[1.]] * batch_size)

        gan.train_on_batch(noise, y2)

```

Əvvəllər müzakirə edildiyi kimi, hər iterasiyada iki mərhələ görə bilərsiniz:

- Birinci mərhələdə, generator-a Gaussian səs-küyü verərək saxta şəkillər istehsal edirik və bu batch-i bərabər sayda həqiqi şəkillərlə birləşdiririk. Hədəflər y1 saxta şəkillər üçün 0, həqiqi şəkillər üçün 1 olaraq təyin edilir. Sonra discriminator-u bu batch üzərində öyrədirik. Xatırlayın ki, discriminator bu mərhələdə trainable-dır, lakin generator-a toxunmuruq.
- İkinci mərhələdə, GAN-a bir az Gaussian səs-küyü veririk. Onun generator-u saxta şəkillər istehsal edəcək, sonra discriminator bu şəkillərin saxta və ya həqiqi olduğunu təxmin etməyə çalışacaq. Bu mərhələdə generator-u yaxşılaşdırmağa çalışırıq, yəni discriminator-un uğursuz olmasını istəyirik: buna görə y2 hədəfləri hamısı 1 olaraq təyin edilir, baxmayaraq ki, şəkillər saxtadır. Bu mərhələdə discriminator *trainable* deyil, buna görə gan modelinin təkmilləşəcək yeganə hissəsi generator-dur.

Hamısı budur! Təlimdən sonra, Gaussian paylanmasından təsadüfi olaraq bəzi codings nümunəsi götürə və onları generator-a verərək yeni şəkillər istehsal edə bilərsiniz:

```

codings = tf.random.normal(shape=[batch_size, codings_size])

generated_images = generator.predict(codings)

```

Yaradılan şəkilləri göstərsəniz (bax Şəkil 17-15), birinci dövrün sonunda onlar artıq (çox səs-küylü) Fashion MNIST şəkillərinə bənzəməyə başlayır.

[Şəkil 17-15] – GAN tərəfindən yaradılan şəkillər, müxtəlif dövrlərdə.

GAN-ların Təliminin Çətinlikləri

Təlim zamanı, generator və discriminator daim bir-birini ağılla keçməyə çalışır, sıfır cəmlə bir oyunda. Təlim irəlilədikcə, oyun oyun nəzəriyyəçilərinin *Nash equilibrium* (Con Nash-ın adı ilə) adlandırdığı bir vəziyyətə çatır: bu, heç bir oyunçunun digər oyunçular öz strategiyalarını dəyişmədiyi halda öz strategiyasını dəyişdirməklə daha yaxşı olmayacağı vəziyyətdir. Məsələn, hamı yolun sol tərəfində sürdükdə Nash equilibrium əldə edilir: heç bir sürücü tək tərəf dəyişən olmaqla daha yaxşı olmaz. Əlbəttə, ikinci mümkün Nash equilibrium var: hamı yolun sağ tərəfində sürdükdə. Fərqli ilkin vəziyyətlər və dinamikalar bir və ya digər tarazlığa gətirib çıxara bilər. Bu nümunədə, tarazlıq əldə edildikdən sonra tək bir optimal strategiya var (hamı ilə eyni tərəfdə sürmək), lakin Nash equilibrium çoxsaylı rəqabət edən strategiyaları əhatə edə bilər (məsələn, yırtıcı ovunu qovalayır, ov qaçmağa çalışır və heç biri strategiyasını dəyişməklə daha yaxşı olmaz).

Bəs GAN-lara necə tətbiq olunur? GAN məqaləsinin müəllifləri bir GAN-ın yalnız tək bir Nash equilibrium-a çatmağa nümayiş etdirdilər: bu, generator-un mükəmməl realistik şəkillər istehsal etdiyi və discriminator-un təxmin etməyə məcbur olduğu (50% həqiqi, 50% saxta) vaxtdır. Bu fakt çox təşviqedicidir: görünür, GAN-ı kifayət qədər uzun müddət öyrətmək kifayətdir və nəhayət bu tarazlığa çatacaq, sizə mükəmməl bir generator verəcəkdir. Təəssüf ki, bu o qədər də sadə deyil: heç nə tarazlığın heç vaxt əldə ediləcəyinə zəmanət vermir.

Ən böyük çətinlik *mode collapse* adlanır: bu, generator-un çıxışlarının tədricən daha az müxtəlif olmasıdır. Bu necə baş verə bilər? Tutaq ki, generator hər hansı digər sinifdən daha inandırıcı ayaqqabılar istehsal etməkdə yaxşılaşır. O, ayaqqabılarla discriminator-u bir az daha aldadacaq və bu, onu daha çox ayaqqabı şəkilləri istehsal etməyə təşviq edəcək. Tədricən, başqa heç nə istehsal etməyi unudacaq. Bu arada, discriminator-un görəcəyi yeganə saxta şəkillər ayaqqabılar olacaq, buna görə o da digər siniflərin saxta şəkillərini ayırmağı unudacaq. Nəhayət, discriminator saxta ayaqqabıları həqiqi olanlardan ayırmağı bacardıqda, generator başqa bir sinifə keçməyə məcbur olacaq. O, sonra köynəklərdə yaxşı ola bilər, ayaqqabıları unudacaq və discriminator da onu izləyəcək. GAN tədricən bir neçə sinif arasında tsikl edə bilər, heç birində həqiqətən çox yaxşı olmaz.

Üstəlik, generator və discriminator daim bir-birinə qarşı itələdiyi üçün, onların parametrləri sonda salınma və qeyri-sabitliyə düşə bilər. Təlim düzgün başlaya bilər, sonra heç bir aydın səbəb olmadan qəflətən ayrılabilir, bu qeyri-sabitliklər səbəbindən. Və bu mürəkkəb dinamiklərə bir çox amillər təsir etdiyi üçün, GAN-lar hiperparametrlərə çox həssasdır: onları incə tənzimləmək üçün çox say sərf etməli ola bilərsiniz. Əslində, buna görə modelləri kompilyasiya edərkən Nadam əvəzinə

RMSProp istifadə etdim: Nadam istifadə edərkən, ağır bir mode collapse ilə qarşılaşdım.

Bu problemlər tədqiqatçıları 2014-cü ildən bəri çox məşğul saxlamışdır: bu mövzuda çoxlu məqalələr nəşr edilmişdir, bəziləri yeni məsrəf funksiyaları (baxmayaraq ki, 2018-ci ildə Google tədqiqatçılarının bir məqaləsi onların effektivliyini sual altına alır) və ya təlimi sabitləşdirmək və ya mode collapse probleminin qarşısını almaq üçün texnikalar təklif etmişdir. Məsələn, *experience replay* adlı populyar bir texnika, generator tərəfindən hər iterasiyada istehsal olunan şəkilləri bir replay buffer-da saxlamaq (tədricən köhnə yaradılmış şəkilləri atmaq) və discriminator-u həqiqi şəkillər plus bu buffer-dən çəkilməmiş saxta şəkillər (cari generator tərəfindən istehsal edilən saxta şəkillər əvəzinə) istifadə edərək öyrətməkdir. Bu, discriminator-un ən son generator çıxışlarına overfitting etməsi şanslarını azaldır. Başqa bir ümumi texnika *mini-batch discrimination* adlanır: o, batch üzrə şəkillərin nə qədər oxşar olduğunu ölçür və bu statistikası discriminator-a təqdim edir, beləliklə o, müxtəliflikdən məhrum olan bütün saxta şəkillər batch-lərini asanlıqla rədd edə bilər. Bu, generator-u daha çox müxtəlif şəkillər istehsal etməyə təşviq edir, mode collapse şansını azaldır. Digər məqalələr sadəcə yaxşı çıxış edən xüsusi arxitekturalar təklif edir.

Qısacası, bu hələ çox aktiv tədqiqat sahəsidir və GAN-ların dinamikası hələ tam başa düşülməyib. Lakin yaxşı xəbər odur ki, böyük irəliləyiş əldə edilmişdir və bəzi nəticələr həqiqətən heyranamızıdır! Beləliklə, gəlin ən uğurlu arxitekturalardan bəzilərini baxaq, ilk növbədə deep convolutional GAN-lar ilə başlayaq, bunlar bir neçə il əvvəl ən müasir vəziyyət idi. Sonra daha yeni (və daha mürəkkəb) iki arxitektura baxacağıq.

Deep Convolutional GANs

Orijinal GAN məqaləsinin müəllifləri convolutional qatlarla təcrübə apardılar, lakin yalnız kiçik şəkillər yaratmağa çalışdılar. Qısa müddət sonra, bir çox tədqiqatçıları daha böyük şəkillər üçün daha dərin convolutional şəbəkələrə əsaslanan GAN-lar qurmağa çalışdılar. Bu, təlim çox qeyri-sabit olduğu üçün çətin oldu, lakin Alec Radford və digərləri nəhayət 2015-ci ilin sonlarında, bir çox müxtəlif arxitektura və hiperparametrləri sınaqdan keçirdikdən sonra uğur qazandılar. Onlar öz arxitekturalarını *deep convolutional GANs* (DCGANs) adlandırdılar. Sabit convolutional GAN-lar qurmaq üçün təklif etdikləri əsas göstərişlər bunlardır:

- Hər hansı pooling qatlarını discriminator-da strided convolutions və generator-da transposed convolutions ilə əvəz edin.
- Həm generator, həm də discriminator-da batch normalization istifadə edin, generator-un çıxış qatı və discriminator-un giriş qatı istisna olmaqla.
- Daha dərin arxitekturalar üçün tam bağlı (fully connected) gizli qatları çıxarın.

- Generator-da çıxış qatından başqa bütün qatlar üçün ReLU aktivasiyasından istifadə edin, çıxış qatı tanh istifadə etməlidir.
- Discriminator-da bütün qatlar üçün leaky ReLU aktivasiyasından istifadə edin.

Bu göstərişlər bir çox hallarda işləyəcək, lakin hər zaman deyil, buna görə hələ də müxtəlif hiperparametrlərlə təcrübə etməli ola bilərsiniz. Əslində, sadəcə təsadüfi toxumu (random seed) dəyişdirmək və eyni modeli yenidən öyrətmək bəzən işləyəcək. Budur Fashion MNIST ilə kifayət qədər yaxşı işləyən kiçik bir DCGAN:

```
codings_size = 100

generator = tf.keras.Sequential([
    tf.keras.layers.Dense(7 * 7 * 128),
    tf.keras.layers.Reshape([7, 7, 128]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2,
padding="same",
                                activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2,
                                padding="same", activation="tanh"),
])

discriminator = tf.keras.Sequential([
    tf.keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="same",
activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="same",
activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

gan = tf.keras.Sequential([generator, discriminator])
```

Generator 100 ölçülü codings qəbul edir, onları 6,272 ölçüyə ($7 \times 7 \times 128$) layihələndirir və nəticəni $7 \times 7 \times 128$ tensoruna formalaşdırır. Bu tensor batch normalization-dan keçir və stride 2 olan transposed convolutional qata qidalanır, bu, onu 7×7 -dən 14×14 -ə böyüdür və dərinliyi 128-dən 64-ə endirir. Nəticə yenidən batch normalization-dan keçir və stride 2 olan başqa bir transposed convolutional qata qidalanır, bu, onu 14×14 -dən 28×28 -ə böyüdür və dərinliyi 64-dən 1-ə endirir. Bu qat tanh aktivasiya funksiyasından istifadə edir, buna görə çıxışlar -1 ilə 1 arasında olacaq. Bu səbəbdən, GAN-ı öyrətməzdən əvvəl, təlim setini eyni diapazona

yenidən miqyaslandırmalıyıq. Biz həmçinin kanal ölçüsünü əlavə etmək üçün onu yenidən formalaşdırmalıyıq:

```
X_train_dcgan = X_train.reshape(-1, 28, 28, 1) * 2. - 1. # reshape and rescale
```

Discriminator, şəkli aşağı ölçüləndirmək üçün max pooling qatları əvəzinə strided convolutions (strides=2) istifadə etməsi istisna olmaqla, binary classification üçün adi CNN-ə çox bənzəyir. Qeyd edək ki, biz leaky ReLU aktivasiya funksiyasından istifadə edirik. Ümumilikdə, DCGAN göstərişlərinə riayət etdik, discriminator-dakı BatchNormalization qatlarını Dropout ilə əvəz etdik; əks halda, təlim bu halda qeyri-sabit idi. Bu arxitekturanı tənzimləməkdən çəkinmeyin: onun hiperparametrlərə, xüsusən iki şəbəkənin nisbi öyrənmə sürətlərinə nə qədər həssas olduğunu görəcəksiniz.

Nəhayət, dataset-i qurmaq və sonra bu modeli kompilyasiya edib öyrətmək üçün əvvəlki kodu istifadə edə bilərik. 50 dövr təlimdən sonra, generator Şəkil 17-16-da göstərilən kimi şəkillər istehsal edir. Hələ mükəmməl deyil, lakin bir çoxu aydın şəkildə müəyyən edilə bilən paltar növləridir.

[Şəkil 17-16] – DCGAN tərəfindən yaradılan şəkillər.

Conditional GANs

Əgər yaradılan şəklin sinfini idarə edə bilmək istəyirsinizsə, bir *conditional GAN* (CGAN) qura bilərsiniz. Fikir sadədir: generator-a Gaussian səs-küyü ilə yanaşı, istədiyiniz sinfin bir-hot vektorunu da qidalandırın və discriminator-a həm şəkli, həm də sinfi qidalandırın. Bu, discriminator-un şəklin verilmiş sinfə uyğun olub-olmadığını yoxlamasına imkan verir. CGAN-lar 2014-cü ildə Mehdi Mirza və Simon Osindero tərəfindən təqdim edilmişdir.

[Şəkil 17-17] – Conditional GAN: generator və discriminator həm səs-küyü, həm də sinif məlumatını qəbul edir.

Progressive Growing of GANs

2018-ci ildə Nvidia tədqiqatçıları Tero Karras və başqaları mühüm bir texnika təklif etdilər: təlimin əvvəlində kiçik şəkillər yaratmaq, sonra həm generator, həm də discriminator-a tədricən convolutional qatlar əlavə edərək daha böyük şəkillər (4×4 , 8×8 , 16×16 , ..., 512×512 , $1,024 \times 1,024$) istehsal etmək. Bu yanaşma stacked autoencoders-in greedy layerwise təliminə bənzəyir. Əlavə qatlar generator-un

sonunda və discriminator-un əvvəlində əlavə olunur və əvvəllər öyrədilmiş qatlar trainable olaraq qalır.

Məsələn, generator-un çıxışları 4×4 -dən 8×8 -ə böyüdüldükdə (bax Şəkil 17-18), mövcud convolutional qata ("Conv 1") bir upsampling qatı (ən yaxın qonşu filtrləməsi istifadə edərək) əlavə edilərək 8×8 feature map-lar istehsal olunur. Bunlar yeni convolutional qata ("Conv 2") qidalanır, o da öz növbəsində yeni bir çıxış convolutional qatına qidalanır. Conv 1-in öyrədilmiş çəkirlərini qırmaqdan qaçmaq üçün, biz iki yeni convolutional qatı tədricən fade-in edirik (Şəkil 17-18-də kəsikli xətlərlə təmsil olunur) və orijinal çıxış qatını fade-out edirik. Son çıxışlar yeni çıxışların (α çəkisi ilə) və orijinal çıxışların ($1 - \alpha$ çəkisi ilə) çəkili cəmidir, α 0-dan 1-ə tədricən artır. Discriminator-a yeni bir convolutional qat əlavə edildikdə (ardınca aşağı ölçüləndirmə üçün ortalama pooling qatı) oxşar fade-in/fade-out texnikası istifadə olunur. Qeyd edək ki, bütün convolutional qatlar "same" padding və stride 1 istifadə edir, buna görə girişlərin hündürlüyünü və enini qoruyur. Bu, orijinal convolutional qatı da əhatə edir, buna görə indi 8×8 çıxışlar istehsal edir (çünki onun girişləri indi 8×8 -dir). Nəhayət, çıxış qatları kernel ölçüsü 1 istifadə edir. Onlar sadəcə girişlərini istədikləri rəng kanallarının sayına (adətən 3) layihələndirir.

[Şəkil 17-18] – Progressive growing: generator-un çıxış ölçüsü 4×4 -dən 8×8 -ə qədər böyüdüldü, yeni qatlar tədricən fade-in edilir.

Bu yanaşma ilə, Nvidia komandası $1,024 \times 1,024$ təsvir ölçülü inanılmaz dərəcədə inandırıcı üz şəkilləri yarada bilən GAN-lar qurmağı bacardı. Onlar həmçinin təlimi sabitləşdirmək üçün bir neçə əlavə texnika tətbiq etdilər:

Mini-batch standard deviation

Discriminator-un sonuna yaxın əlavə edilən bir qat. Girişlərin hər bir mövqeyi üçün, o, bütün kanallar və batch-dəki bütün instanslar üzərində standart sapmanı hesablayır ($S = \text{tf.math.reduce_std}(\text{inputs}, \text{axis}=[0, -1])$). Bu standart sapmalar daha sonra bütün nöqtələr üzərində ortalama alınaraq tək bir qiymət əldə edilir ($v = \text{tf.reduce_mean}(S)$). Nəhayət, batch-dəki hər bir instansa əlavə bir feature map əlavə edilir və hesablanmış qiymətlə doldurulur ($\text{tf.concat}([\text{inputs}, \text{tf.fill}([\text{batch_size}, \text{height}, \text{width}, 1], v)], \text{axis}=-1)$). Bu necə kömək edir? Yaxşı, əgər generator az müxtəlifliyə malik şəkillər istehsal edərsə, discriminator-da feature map-lar arasında kiçik bir standart sapma olacaq. Bu qat sayəsində, discriminator bu statistikaya asanlıqla çıxış əldə edəcək, bu da onun çox az müxtəliflik istehsal edən bir generator tərəfindən aldadılması ehtimalını azaldacaq. Bu, generator-u daha müxtəlif çıxışlar istehsal etməyə təşviq edəcək, mode collapse riskini azaldacaq.

Equalized learning rate

Bütün çəkiləri He ilkinləşdirməsi əvəzinə ortalama 0 və standart sapma 1 olan Gaussian paylanmasından istifadə edərək ilkinləşdirir. Lakin çəkilər iş vaxtı (yəni qat hər dəfə icra edildikdə) He ilkinləşdirməsində olduğu kimi eyni amillə aşağı miqyaslandırılır: onlar $2 / \sqrt{n_inputs}$ -ə bölünür, burada n_inputs qata olan girişlərin sayıdır. Məqalə göstərdi ki, bu texnika RMSProp, Adam və ya digər adaptiv qradiyent optimizatorlarından istifadə edərkən GAN-ın performansını əhəmiyyətli dərəcədə yaxşılaşdırdı. Həqiqətən, bu optimizatorlar qradiyent yeniləmələrini təxmin edilən standart sapmalarına görə normallaşdırır (bax Fəsil 11), buna görə daha böyük dinamik diapazona malik parametrlər daha uzun təlim tələb edəcək, kiçik dinamik diapazona malik parametrlər isə çox sürətli yenilənə bilər, qeyri-sabitliklərə səbəb olur. Çəkiləri yalnız ilkinləşdirmə zamanı deyil, modelin özünün bir hissəsi kimi yenidən miqyaslandırmaqla, bu yanaşma bütün parametrlərin təlim boyunca eyni dinamik diapazona malik olmasını təmin edir, beləliklə hamısı eyni sürətlə öyrənir. Bu, həm təlimi sürətləndirir, həm də sabitləşdirir.

Pixelwise normalization layer

Generator-da hər bir convolutional qatdan sonra əlavə edilir. O, eyni şəkildə və eyni yerdə, lakin bütün kanallar üzərində bütün aktivasiyalara əsaslanaraq hər bir aktivasiyanı normallaşdırır (ortalama kvadrat aktivasiyanın kvadrat kökünə bölməklə). TensorFlow kodunda, bu, `inputs / tf.sqrt(tf.reduce_mean(tf.square(X), axis=-1, keepdims=True) + 1e-8)` dir (1e-8 hamarlaşdırma termini sıfıra bölmənin qarşısını almaq üçün lazımdır). Bu texnika, generator və discriminator arasında həddindən artıq rəqabət səbəbindən aktivasiyalarda partlayışların qarşısını alır.

Bütün bu texnikaların birləşməsi müəlliflərə son dərəcə inandırıcı yüksək dəqiqlikli üz şəkilləri yaratmağa imkan verdi. Bəs "inandırıcı" nəyə deyirik? Qiymətləndirmə GAN-larla işləyərkən böyük çətinliklərdən biridir: yaradılan şəkillərin müxtəlifliyini avtomatik qiymətləndirmək mümkün olsa da, onların keyfiyyətini mühakimə etmək daha çətin və subyektiv bir işdir. Bir texnika, insan qiymətləndiricilərindən istifadə etməkdir, lakin bu bahalı və vaxt aparandır. Beləliklə, müəlliflər yaradılan şəkillərin yerli şəkil strukturunun təlim şəkilləri ilə oxşarlığını hər bir miqyasda ölçməyi təklif etdilər. Bu fikir onları daha bir qabaqcıl yeniliyə gətirdi: StyleGAN-lar.

StyleGANs

Yüksək dəqiqlikli şəkil yaratmada ən müasir vəziyyət, eyni Nvidia komandası tərəfindən 2018-ci il məqaləsində təqdim edilən məşhur StyleGAN arxitekturası ilə bir daha irəlilədi. Müəlliflər generator-da *style transfer* texnikalarından istifadə etdilər ki, yaradılan şəkillər hər miqyasda təlim şəkilləri ilə eyni yerli strukturuna malik olsun, bu da yaradılan şəkillərin keyfiyyətini böyük dərəcədə yaxşılaşdırdı. Discriminator və loss

funksiyası dəyişdirilmədi, yalnız generator dəyişdirildi. Bir StyleGAN generator-u iki şəbəkədən ibarətdir (bax Şəkil 17-19):

Mapping network

z Səkkiz qatlı bir MLP, latent təsvirləri (yəni codings) w vektoruna xəritələşdirir. Bu vektor daha sonra çoxsaylı *affine transformations* (yəni aktivasiya funksiyası olmayan Dense qatları, Şəkil 17-19-da "A" qutuları ilə təmsil olunur) vasitəsilə göndərilir və çoxsaylı vektorlar istehsal edir. Bu vektorlar yaradılan şəklin müxtəlif səviyyələrdəki stilini idarə edir, incə toxumadan (məsələn, saç rəngi) yüksək səviyyəli xüsusiyyətlərə qədər (məsələn, böyük və ya uşaq). Qısacası, mapping network codings-i çoxsaylı style vektorlarına xəritələşdirir.

Synthesis network

Şəkillərin yaradılmasına cavabdehdir. Onun sabit öyrənilmiş girişi var (aydın olmaq üçün, bu giriş təlimdən sonra sabit olacaq, lakin təlim zamanı backpropagation ilə dəyişdirilməyə davam edir). O, bu girişi əvvəlki kimi çoxsaylı convolutional və upsampling qatlarından keçirir, lakin iki fərqlə. Birincisi, girişə və convolutional qatların bütün çıxışlarına (aktivasiya funksiyasından əvvəl) bəzi səs-küy əlavə olunur. İkincisi, hər bir səs-küy qatından sonra bir *adaptive instance normalization* (AdaIN) qatı gəlir: o, hər bir feature map-ı müstəqil olaraq standartlaşdırır (feature map-ın ortalamasını çıxarır standart sapmasına bölməklə), sonra style vektorundan istifadə edərək hər bir feature map-ın miqyasını və offsetini təyin edir (style vektoru hər bir feature map üçün bir miqyas və bir offset ehtiva edir).

[Şəkil 17-19] – StyleGAN generator: mapping network w vektorlarını yaradır, synthesis network şəkli yaradır, səs-küy və AdaIN qatları ilə.

AdaIN əməliyyatının məqsədi, style vektorunun şəklin stilini idarə etməsinə imkan verməkdir. Mapping network müxtəlif səviyyələr üçün bir neçə style vektoru istehsal etdiyi üçün, bu, generator-a şəklin ayrı-ayrı aspektlərini müstəqil şəkildə idarə etməyə imkan verir. Məsələn, müəyyən bir AdaIN qatı saç rəngini idarə edə bilər, digəri isə üzün formasını.

Style generator-un niyə səs-küy qatlarına ehtiyacı var? Çünki təlim zamanı generator deterministikdir: eyni codings eyni şəkli istehsal edəcək. Lakin yaradılan şəklin hər bir pikselində təsadüfi dəyişkənlik olmalıdır (məsələn, saçların dəqiq yerləşməsi). Generator-un özü psevdorandom səs-küy istehsal etməyə çalışsaydı, əlavə hesablama tələb edərdi və qradientlərin geri axmasına mane olardı. Həmçinin, psevdorandom səs-küy o qədər də inandırıcı görünməyə bilər, daha çox vizual artefaktlara səbəb olar. Üstəlik, generator-un çəkirlərinin bir hissəsi psevdorandom səs-küy istehsalına həsr edilmiş olardı, bu da yenə israfçıdır. Əlavə səs-küy girişləri ilə

bütün bu problemlər aradan qalxır; GAN təqdim olunan səs-küydən istifadə edərək şəklin hər bir hissəsinə düzgün miqdarda stoxastiklik əlavə edə bilər.

Əlavə edilən səs-küy hər səviyyə üçün fərqlidir. Hər bir səs-küy girişi, Gaussian səs-küyü ilə dolu tək bir feature map-dən ibarətdir, bu, bütün feature map-lara (verilmiş səviyyənin) yayılır və əlavə edilməzdən əvvəl öyrənilmiş per-feature miqyas amillərindən istifadə edərək miqyaslandırılır (bu, Şəkil 17-19-da "B" qutuları ilə təmsil olunur).

Nəhayət, StyleGAN *mixing regularization* (və ya *style mixing*) adlanan bir texnika istifadə edir, burada yaradılan şəkillərin bir faizi iki fərqli codings istifadə edərək istehsal olunur. Xüsusilə, codings c_1 və c_2 mapping network vasitəsilə göndərilir, iki style vektoru w_1 və w_2 verir. Sonra synthesis network ilk səviyyələr üçün w_1 stilləri və qalan səviyyələr üçün w_2 stilləri əsasında bir şəkil yaradır. Kəsmə səviyyəsi təsadüfi seçilir. Bu, şəbəkənin bitişik səviyyələrdəki stillərin əlaqəli olduğunu fərz etməsinin qarşısını alır, bu da GAN-da lokalliyi təşviq edir, yəni hər bir style vektoru yaradılan şəkildə yalnız məhdud sayda xüsusiyyətlərə təsir edir.

GAN-ların o qədər geniş müxtəlifliyi var ki, onların hamısını əhatə etmək üçün tam bir kitab lazımdır. Ümid edirəm ki, bu giriş sizə əsas fikirləri və ən əsası, daha çox öyrənmək istəyini verdi. Öz GAN-ınızı tətbiq edin və əvvəlcə öyrənməkdə çətinlik çəkirsə, ruhdan düşməyin: təəssüf ki, bu normaldır və onu işləmək üçün kifayət qədər səbr tələb olunacaq, lakin nəticə buna dəyər. Bir tətbiq detallığı ilə mübarizə aparırsınızsa, baxa biləcəyiniz çoxlu Keras və ya TensorFlow tətbiqləri var. Əslində, əgər yalnız tez bir zamanda heyranəmiz nəticələr əldə etmək istəyirsinizsə, əvvəlcədən öyrədilmiş bir modeldən istifadə edə bilərsiniz (məsələn, Keras üçün əvvəlcədən öyrədilmiş StyleGAN modelləri mövcuddur).

İndi autoencoders və GAN-ları nəzərdən keçirdiyimizə görə, son bir arxitektura növünə baxaq: diffusion modelləri.

Diffusion Modelləri

Diffusion modellərinin arxasındakı fikirlər uzun illərdir mövcuddur, lakin onlar ilk dəfə 2015-ci ildə Stanford Universiteti və UC Berkeley-dən Jascha Sohl-Dickstein və digərlərinin məqaləsində müasir formada rəsmiləşdirilmişdir. Müəlliflər termodinamikadan alətləri bir diffusion prosesini modelləşdirmək üçün tətbiq etdilər, bir damla südün çay fincanında diffuziyasına bənzər. Əsas fikir, modeli tərs prosesi öyrənmək üçün öyrətməkdir: tam qarışıq vəziyyətdən başlayaraq, tədricən südü çaydan "ayırmaq". Bu fikirdən istifadə edərək, şəkil yaratmada ümidverici nəticələr əldə etdilər, lakin o zaman GAN-lar daha inandırıcı şəkillər istehsal etdiyi üçün, diffusion modelləri o qədər diqqət çəkmədi.

Sonra, 2020-ci ildə, yenə UC Berkeley-dən Jonathan Ho və digərləri, yüksək dərəcədə realistik şəkillər yarada bilən bir diffusion modeli qurmağı bacardılar, onlar buna *denoising diffusion probabilistic model* (DDPM) adını verdilər. Bir neçə ay sonra, 2021-ci ildə OpenAI tədqiqatçıları Alex Nichol və Prafulla Dhariwal tərəfindən bir məqalə DDPM arxitekturasını təhlil etdi və DDPM-lərin nəhayət GAN-ları üstələməsinə imkan verən bir neçə təkmilləşdirmə təklif etdi: DDPM-lər təlim etmək üçün GAN-lardan daha asan olmaqla yanaşı, yaradılan şəkillər daha müxtəlif və daha yüksək keyfiyyətlidir. DDPM-lərin əsas çatışmazlığı, GAN-lar və ya VAE-lər ilə müqayisədə şəkillər yaratmaq üçün çox uzun vaxt tələb etmələridir, görəcəyiniz kimi.

Bəs DDPM tam olaraq necə işləyir? Tutaq ki, siz bir pişik şəkli ilə başlayırsınız (Şəkil 17-20-də görəcəyiniz kimi), qeyd olunur x_0 və hər zaman addımında t şəkilə bir az Gaussian səs-küyü əlavə edirsiniz, ortalama 0 və dispersiya βt ilə. Bu səs-küy hər piksel üçün müstəqildir: biz buna *izotrop* deyirik. Siz əvvəlcə x_1 şəklini, sonra x_2 -ni və s. alırsınız, ta ki pişik səs-küy tərəfindən tamamilə gizlədilənədək, görünməz olana qədər. Son zaman addımı T kimi qeyd olunur. Orijinal DDPM məqaləsində müəlliflər $T = 1,000$ istifadə etdilər və dispersiya βt -ni elə planlaşdırdılar ki, pişik signalı 0 və T zaman addımları arasında xətti olaraq olsun. Təkmilləşdirilmiş DDPM məqaləsində T 4,000-ə qaldırıldı və dispersiya planı başlanğıcda və sonunda daha yavaş dəyişmək üçün tənzimləndi. Qısacası, biz tədricən pişiyi səs-küyə batırıq: buna *forward process* deyilir.

[Şəkil 17-20] – Forward diffusion prosesi: orijinal şəkil tədricən səs-küyə batırılır.

Forward prosesdə nə qədər çox Gaussian səs-küyü əlavə etsək, piksel dəyərlərinin paylanması bir o qədər Gaussian olur. Bir vacib detallı buraxmışam: piksel dəyərləri hər addımda $1 - \beta t$ amili ilə bir az yenidən miqyaslandırılır. Bu, piksel dəyərlərinin ortalamasının tədricən 0-a yaxınlaşmasını təmin edir, çünki miqyaslandırma amili 1-dən bir az kiçikdir (bir ədədi 0.99-a təkrar vurmağı təsəvvür edin). O, həmçinin dispersiyanın tədricən 1-ə yığılmasını təmin edir. Bunun səbəbi, piksel dəyərlərinin standart sapması da $1 - \beta t$ ilə miqyaslandırılır, buna görə dispersiya $1 - \beta t$ ilə miqyaslandırılır (yəni miqyaslandırma amilinin kvadratı). Lakin dispersiya 0-a qədər kiçilə bilməz, çünki biz hər addımda dispersiya βt olan Gaussian səs-küyü əlavə edirik. Və Gaussian paylanmalarını cəmləyəndə dispersiyalar toplandığı üçün, dispersiyanın yalnız $1 - \beta t + \beta t = 1$ -ə yığıla biləcəyini görə bilərsiniz.

Forward diffusion prosesi Tənlik 17-5-də xülasə edilmişdir. Bu tənlik forward proses haqqında sizə yeni heç nə öyrətməyəcək, lakin bu tip riyazi notasiyanı başa düşmək faydalıdır, çünki ML məqalələrində tez-tez istifadə olunur. Bu tənlik, x_t -nin x_{t-1} verildikdə ehtimal paylanmasını q Gaussian paylanması kimi təyin edir, ortalama x_{t-1} dəfə miqyaslandırma amili, və kovariasiya matrisi βI -yə bərabərdir. Bu, I eynilik matrisi βt ilə vurulmuşdur, yəni səs-küy dispersiya βt ilə izotopdur.

Tənlik 17-5. Forward diffusion prosesinin ehtimal paylanması q

$$q(x_t | x_{t-1}) = N(\sqrt{1-\beta_t} x_{t-1}, \beta_t I)$$

Maraqlıdır ki, forward proses üçün bir qısa yol var: x_t şəklini x_0 verildikdə, əvvəlcə $x_1, x_2, \dots, x_{t-1}$ hesablamadan nümunə götürmək mümkündür. Həqiqətən, bir neçə Gaussian paylanmasının cəmi də Gaussian paylanması olduğundan, bütün səs-küy Tənlik 17-6 istifadə edərək bir dəfəyə əlavə edilə bilər. Bu, istifadə edəcəyimiz tənlikdir, çünki daha sürətlidir.

Tənlik 17-6. Forward diffusion prosesi üçün qısa yol

$$q(x_t | x_0) = N(\sqrt{\alpha_t} x_0, (1 - \alpha_t) I)$$

Bizim məqsədımız, əlbəttə, pişikləri səs-küydə boğmaq deyil. Əksinə, biz səs-küyü təmizləmək və orijinal şəkli bərpa etmək istəyirik. Bu, *reverse process* -dir. Məqsəd, tərs prosesi öyrənməkdir: x_T tam səs-küydən başlayaraq, tədricən səs-küyü çıxararaq x_0 əldə etmək. Tərs proses x_{t-1} -in x_t verildikdə paylanmasıdır, $p_\theta(x_{t-1} | x_t)$ ilə təmsil olunur (şəkil 17-20-də aşağı oxlar). Əgər β_t çox kiçikdirsə, bu tərs paylanma da Gaussian-a yaxındır, lakin onun ortalama və standart sapmasını bilmirik. Onları təxmin etmək üçün bir neyron şəbəkəsi öyrədirik.

Orijinal DDPM məqaləsində müəlliflər bir model öyrətdilər ki, x_t və t verildikdə, əlavə edilmiş səs-küyü proqnozlaşdırır. Bu, x_{t-1} əldə etmək üçün Tənlik 17-8-i tətbiq etməyə imkan verir. Modeli öyrətmək üçün, bizə hər bir batch üçün səs-küylü şəkillər və onlara əlavə edilmiş səs-küy lazımdır. Bunları təlim setindən orijinal şəkillər götürüb, onlara forward prosesi tətbiq etməklə yarada bilərik.

İndi təlim məlumatlarını hazırlayaq. Fashion MNIST şəkillərini götürəcəyik, onları -1 ilə 1 arasına miqyaslandıracağıq, sonra hər bir şəkil üçün təsadüfi bir zaman addımı t seçəcəyik (1 -dən T -yə qədər) və ona uyğun səs-küy əlavə edəcəyik. Biz həmçinin, α_t dəyərlərini əvvəlcədən hesablayacağıq.

Aşağıdakı kod, hər bir batch üçün səs-küylü şəkillər və onlara uyğun səs-küyü hazırlayan bir funksiyadır:

```
T = 1000
beta = tf.linspace(1e-4, 0.02, T)
alpha = 1 - beta
alpha_cumprod = tf.math.cumprod(alpha, axis=0)

def prepare_batch(X):
    X = tf.cast(X[..., tf.newaxis], tf.float32) * 2 - 1 # scale from -1 to
+1
    X_shape = tf.shape(X)
```

```

        t = tf.random.uniform([X_shape[0]], minval=1, maxval=T + 1,
dtype=tf.int32)
        alpha_cm = tf.gather(alpha_cumprod, t)
        alpha_cm = tf.reshape(alpha_cm, [X_shape[0]] + [1] * (len(X_shape) -
1))
        noise = tf.random.normal(X_shape)
        return {
            "X_noisy": alpha_cm ** 0.5 * X + (1 - alpha_cm) ** 0.5 * noise,
            "time": t,

    }, noise

```

Bu kodu nəzərdən keçirək:

- Sadəlik üçün Fashion MNIST istifadə edəcəyik, buna görə funksiya əvvəlcə kanal oxu əlavə etməlidir. Həmçinin piksel dəyərlərini -1 -dən 1 -ə miqyaslandırmaq kömək edəcək, beləliklə son Gaussian paylanmasına (ortalama 0 , dispersiya 1) daha yaxın olar.
- Sonra, funksiya t yaradır, batch-dəki hər bir şəkil üçün 1 ilə T arasında təsadüfi bir zaman addımını ehtiva edən vektor.
- Sonra, `tf.gather()` istifadə edərək t vektorundakı hər bir zaman addımı üçün `alpha_cumprod` dəyərini alır. Bu, hər bir şəkil üçün bir α_t dəyəri ehtiva edən `alpha_cm` vektorunu verir.
- Növbəti sətir `alpha_cm`-ni `[batch size]` -dən `[batch size, 1, 1, 1]` -ə yenidən formalaşdırır. Bu, `alpha_cm`-nin batch X ilə broadcast edilə bilməsini təmin etmək üçün lazımdır.
- Sonra, ortalama 0 və dispersiya 1 olan bəzi Gaussian səs-küyü yaradıırıq.
- Nəhayət, Tənlik 17-6-nı tətbiq edərək diffusion prosesini şəkillərə tətbiq edirik. Qeyd edək ki, $x ** 0.5$, x -in kvadrat kökünə bərabərdir. Funksiya girişləri və hədəfləri ehtiva edən bir tuple qaytarır. Girişlər səs-küylü şəkilləri və onları yaratmaq üçün istifadə edilən zaman addımlarını ehtiva edən bir Python dict kimi təmsil olunur. Hədəflər, hər bir şəkli yaratmaq üçün istifadə edilən Gaussian səs-küyüdür.

QEYD

Bu quruluşla, model giriş şəklindən çıxarılmalı olan səs-küyü proqnozlaşdıracaqdır. Niyə birbaşa orijinal şəkli proqnozlaşdırmırıq? Müəlliflər bunu sınaqda: sadəcə işləmir.

Sonra, hər bir batch-ə `prepare_batch()` funksiyasını tətbiq edəcək bir təlim dataset-i və validasiya seti yaradacağıq. Əvvəlki kimi, `X_train` və `X_valid` piksel dəyərləri 0 ilə 1 arasında olan Fashion MNIST şəkillərini ehtiva edir:

```
def prepare_dataset(X, batch_size=32, shuffle=False):
    ds = tf.data.Dataset.from_tensor_slices(X)
    if shuffle:
        ds = ds.shuffle(buffer_size=10_000)
    return ds.batch(batch_size).map(prepare_batch).prefetch(1)

train_set = prepare_dataset(X_train, batch_size=32, shuffle=True)

valid_set = prepare_dataset(X_valid, batch_size=32)
```

İndi faktiki diffusion modelini qurmağa hazırıq. O, səs-küylü şəkillər və zaman addımlarını giriş kimi qəbul edən və giriş şəkillərindən çıxarılaçaq səs-küyü proqnozlaşdıran istənilən model ola bilər:

```
def build_diffusion_model():
    X_noisy = tf.keras.layers.Input(shape=[28, 28, 1], name="X_noisy")
    time_input = tf.keras.layers.Input(shape=[], dtype=tf.int32,
name="time")
    [...] # build the model based on the noisy images and the time steps
    outputs = [...] # predict the noise (same shape as the input images)

    return tf.keras.Model(inputs=[X_noisy, time_input], outputs=[outputs])
```

DDPM müəllifləri modifikasiya edilmiş U-Net arxitekturasından istifadə etdilər, bu, Fəsil 14-də semantic segmentation üçün müzakirə etdiyimiz FCN arxitekturası ilə bir çox oxşarlıqlara malikdir: o, giriş şəkillərini tədricən aşağı ölçüləndirən, sonra tədricən yenidən böyüdən bir convolutional neural network-dur, aşağı ölçüləndirmə hissəsinin hər səviyyəsindən yuxarı ölçüləndirmə hissəsinin müvafiq səviyyəsinə skip connections ilə. Zaman addımlarını nəzərə almaq üçün, onlar transformer arxitekturasındakı mövqe kodlaşdırmaları ilə eyni texnikadan istifadə edərək onları kodlaşdırdılar (bax Fəsil 16). U-Net arxitekturasının hər səviyyəsində, bu zaman kodlaşdırmalarını Dense qatlarından keçirdilər və U-Net-ə qidalandırdılar. Nəhayət, onlar müxtəlif səviyyələrdə multi-head attention qatlarından da istifadə etdilər. Əsas tətbiq üçün bu fəslin notebook-na baxın və ya <https://homl.info/ddpmcode> ünvanında rəsmi tətbiqə baxın: bu, TF 1.x əsaslıdır və köhnəlmişdir, lakin olduqca oxunaqlıdır.

İndi modeli normal şəkildə öyrədə bilərik. Müəlliflər qeyd etdilər ki, MAE loss-dan istifadə MSE-dən daha yaxşı işləyirdi. Siz həmçinin Huber loss-dan da istifadə edə bilərsiniz:

```
model = build_diffusion_model()
model.compile(loss=tf.keras.losses.Huber(), optimizer="nadam")

history = model.fit(train_set, validation_data=valid_set, epochs=100)
```

Model öyrədildikdən sonra, ondan yeni şəkillər yaratmaq üçün istifadə edə bilərsiniz. Təəssüf ki, tərs diffusion prosesində qısa yol yoxdur, buna görə x_T -ni ortalama 0 və dispersiya 1 olan Gaussian paylanmasından təsadüfi nümunə götürməli, sonra onu modelə ötürərək səs-küyü proqnozlaşdırmalı; onu Tənlik 17-8 istifadə edərək şəkildən çıxarmalı və x_{T-1} əldə etməlisiniz. x_0 əldə edənə qədər prosesi 3,999 dəfə daha təkrarlayın: əgər hər şey yaxşı gedərsə, o, adi bir Fashion MNIST şəkli kimi görünməlidir!

Tənlik 17-8. Diffusion prosesində tərs istiqamətdə bir addım

$$x_{t-1} = (1 / \sqrt{\alpha_t}) (x_t - (\beta_t / \sqrt{(1 - \bar{\alpha}_t)}) \epsilon_{\theta}(x_t, t)) + \sqrt{\beta_t} z$$

Bu tənlikdə, $\epsilon_{\theta}(x_t, t)$, giriş şəkli x_t və zaman addımı t verildikdə model tərəfindən proqnozlaşdırılan səs-küyü təmsil edir. θ model parametrlərini təmsil edir. Üstəlik, z ortalama 0 və dispersiya 1 olan Gaussian səs-küyüdür. Bu, tərs prosesi stoxastik edir: onu bir neçə dəfə işlətsəniz, müxtəlif şəkillər alacaqsınız.

Gəlin bu tərs prosesi tətbiq edən bir funksiya yazaq və onu bir neçə şəkil yaratmaq üçün çağıraraq:

```
def generate(model, batch_size=32):
    X = tf.random.normal([batch_size, 28, 28, 1])
    for t in range(T, 0, -1):
        noise = (tf.random.normal if t > 1 else tf.zeros)(tf.shape(X))
        X_noise = model({"X_noisy": X, "time": tf.constant([t] *
batch_size)})
        X = (1 / alpha[t] ** 0.5) * (X - (beta[t] / (1 - alpha_cumprod[t]
** 0.5) * X_noise) + (beta[t] ** 0.5) * noise

    return X
```

[Şəkil 17-21] – DDPM tərəfindən yaradılan şəkillər.

DDPM-lər təlim etmək üçün asan olsa da, yaradılan şəkillər üçün çox uzun müddət tələb olunur. 2021-ci ilin sonlarında, LMU Munich-dən Robin Rombach və digərləri *latent diffusion* (LD) adlanan bir yanaşma təklif etdilər. Onlar diffusion prosesini piksel fəzasında deyil, aşağı ölçülü bir latent fəzada həyata keçirdilər (buna görə latent diffusion). LD, bir autoencoder (adətən VAE) istifadə edərək şəkilləri aşağı ölçülü latent fəzaya sıxır, sonra diffusion modelini bu latent fəzada tətbiq edir və nəhayət, yaradılan latent təsviri autoencoder-in decoder-i ilə decode edir. Bu yanaşma ilə, yaradılan şəkillər üçün tələb olunan vaxt GAN-larla rəqabət edə bilər, çünki diffusion prosesi daha kiçik ölçülü məkanlarda baş verir və nəticədə yaranan şəkillər yüksək dəqiqliklidir. Qısacası, həm GAN-ların sürəti, həm də DDPM-lərin keyfiyyəti birləşdirilir.

Latent diffusion modellərinin bir digər böyük üstünlüyü onların çox güclü keşilləndirilməsidir. Siz modelə əlavə girişlər (məsələn, mətn promptları) verə bilərsiniz və bu, modelin tamamilə yeni növ şəkillər yaratmasına imkan verir. Bu, DALL·E 2 və Imagen kimi modellərin əsasını təşkil edir.

2022-ci ilin avqustunda, LMU Munich və StabilityAI, Runway kimi bir neçə şirkət arasında əməkdaşlıqla, *Stable Diffusion* adlı güclü bir əvvəlcədən öyrədilmiş latent diffusion modeli açıq qaynaq edildi. 2022-ci ilin sentyabrında, TensorFlow-a port edildi və KerasCV-yə daxil edildi. İndi hər kəs müntəzəm bir laptopda belə, saniyələr ərzində, pulsuz olaraq heyranəmiz şəkillər yarada bilər. İmkanlar sonsuzdur!

Növbəti fəsildə biz deep learning-in tamamilə fərqli bir qoluna keçəcəyik: deep reinforcement learning.

Məşqlər

1. Autoencoders-in əsas tapşırıqları hansılardır?
2. Tutaq ki, bir təsnifatçı öyrətmək istəyirsiniz və bol etiketsiz təlim məlumatınız var, lakin yalnız bir neçə min etiketli instansınız var. Autoencoders necə kömək edə bilər? Necə davam edərdiniz?
3. Əgər bir autoencoder girişləri mükəmməl rekonstruksiya edərsə, bu, mütləq yaxşı bir autoencoder-dirmi? Autoencoder-in performansını necə qiymətləndirə bilərsiniz?
4. Undercomplete və overcomplete autoencoders nədir? Həddindən artıq undercomplete autoencoder-in əsas riski nədir? Bəs overcomplete autoencoder-in əsas riski?
5. Stacked autoencoder-də çəkirləri necə bağlamaq olar? Bunu etməyin məqsədi nədir?
6. Generative model nədir? Bir növ generative autoencoder adlandırı bilərsinizmi?
7. GAN nədir? GAN-ların parlaya biləcəyi bir neçə tapşırıq adlandırı bilərsinizmi?
8. GAN-ları öyrədərkən əsas çətinliklər hansılardır?
9. Diffusion modelləri nədə yaxşıdır? Onların əsas məhdudiyyəti nədir?
10. Bir denoising autoencoder istifadə edərək bir şəkil təsnifatçısını pretraining etməyə çalışın. MNIST (ən sadə seçim) və ya daha böyük bir problem istəyirsinizsə, CIFAR10 kimi daha mürəkkəb bir şəkil dataset-indən istifadə edə bilərsiniz. İstifadə etdiyiniz dataset-dən asılı olmayaraq, bu addımları izləyin:
 - a. Dataset-i təlim seti və test setinə bölün. Tam təlim seti üzərində dərin bir denoising autoencoder öyrədin.

- b. Şəkillərin kifayət qədər yaxşı rekonstruksiya edildiyini yoxlayın. Coding qatındakı hər bir neyronu ən çox aktivləşdirən şəkilləri vizuallaşdırın.
 - c. Autoencoder-in aşağı qatlarını təkrar istifadə edən bir təsnifat DNN qurun. Onu təlim setindən yalnız 500 şəkil istifadə edərək öyrədin. Pretraining ilə və ya olmadan daha yaxşı performans göstərirmi?
11. Seçdiyiniz şəkil dataset-i üzərində bir variational autoencoder öyrədin və ondan şəkillər yaratmaq üçün istifadə edin. Alternativ olaraq, sizi maraqlandıran etiketsiz bir dataset tapmağa və yeni nümunələr yarada biləcəyinizi görməyə çalışın.
 12. Seçdiyiniz şəkil dataset-i üzərində bir DCGAN öyrədin və ondan şəkillər yaratmaq üçün istifadə edin. Experience replay əlavə edin və bunun kömək edib-etmədiyinə baxın. Onu yaradılan sinfi idarə edə biləcəyiniz bir conditional GAN-a çevirin.
 13. KerasCV-nin əla Stable Diffusion tutorial-nı nəzərdən keçirin və kitab oxuyan bir salamanderin gözəl bir rəsmini yaradın. Ən yaxşı rəsminizi Twitter-də yerləşdirsəniz, zəhmət olmasa məni @aureliengeron olaraq qeyd edin. Yaradıcılığınızı görmək istərdim!

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.